Transfer Learning with a Pre-trained CNN

Framework: TensorFlow / Keras | **Pre-trained model:** MobileNetV2 | **Dataset:** CIFAR-10

Aim

Demonstrate transfer learning by loading a pre-trained CNN, freezing its learned layers, adding and training a custom classifier, and fine-tuning selected layers.

Important: CIFAR-10 has image-level class labels, not bounding boxes. This notebook demonstrates transfer learning for image classification, a common building block for object detection. Actual object detection requires bounding-box annotations and a detector such as SSD, Faster R-CNN, or YOLO.

a. Load a pre-trained CNN and access the dataset

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10

print("TensorFlow version:", tf.__version__)

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
class_names = ["airplane", "automobile", "bird", "cat", "deer",
               "dog", "frog", "horse", "ship", "truck"]

# Use a subset to make the practical faster on Colab
x_train, y_train = x_train[:10000], y_train[:10000].flatten()
x_test, y_test = x_test[:2000], y_test[:2000].flatten()

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)
```

```
TensorFlow version: 2.21.0
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 2417s 14us/step
Training data: (10000, 32, 32, 3)
Testing data: (2000, 32, 32, 3)
```

b. Freeze pre-trained CNN layers

```
# MobileNetV2 has pre-trained ImageNet weights.
# The original classification head is removed.
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(96, 96, 3),
    include_top=False,
    weights="imagenet"
)
base_model.trainable = False
print("Pre-trained MobileNetV2 loaded; all base layers frozen.")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Pre-trained MobileNetV2 loaded; all base layers frozen.
```

c. Add a custom classifier and compile the model

```
inputs = layers.Input(shape=(32, 32, 3), name="input_image")
x = layers.RandomFlip("horizontal")(inputs)
x = layers.Resizing(96, 96)(x)

# MobileNetV2 preprocessing expects input pixels scaled to approximately [-1, 1]
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
features = base_model(x, training=False)

x = layers.GlobalAveragePooling2D()(features)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(10, activation="softmax")(x)
```

```
model = models.Model(inputs, outputs, name="TransferLearningModel")
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model.summary()
```

Model: "TransferLearningModel"

Layer (type)	Output Shape	Param #
input_image (InputLayer)	(None, 32, 32, 3)	0
random_flip (RandomFlip)	(None, 32, 32, 3)	0
resizing (Resizing)	(None, 96, 96, 3)	0
true_divide (TrueDivide)	(None, 96, 96, 3)	0
subtract (Subtract)	(None, 96, 96, 3)	0
mobilenetv2_1.00_96 (Functional)	(None, 3, 3, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense (Dense)	(None, 128)	163,968
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 2,423,242 (9.24 MB)

d. Train the custom classifier

```
history_initial = model.fit(
    x_train, y_train,
    validation_split=0.2,
    epochs=5,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 82s 307ms/step - accuracy: 0.7200 - loss: 0.8191 - val_accuracy: 0.8045 - val_loss: 0.5710
Epoch 2/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 76s 303ms/step - accuracy: 0.8278 - loss: 0.5028 - val_accuracy: 0.8230 - val_loss: 0.5167
Epoch 3/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 68s 273ms/step - accuracy: 0.8631 - loss: 0.4032 - val_accuracy: 0.8445 - val_loss: 0.4657
Epoch 4/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 68s 273ms/step - accuracy: 0.8810 - loss: 0.3458 - val_accuracy: 0.8475 - val_loss: 0.4610
Epoch 5/5
250/250 ━━━━━━━━━━━━━━━━━━━━ 81s 271ms/step - accuracy: 0.8955 - loss: 0.2990 - val_accuracy: 0.8570 - val_loss: 0.4761
```

e. Fine-tune the model by unfreezing selected upper layers

```
# Unfreeze the top 20% of MobileNetV2 layers; keep earlier layers frozen
base_model.trainable = True
fine_tune_at = int(len(base_model.layers) * 0.8)

for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

# Keep Batch Normalization layers frozen for stable fine-tuning
for layer in base_model.layers:
    if isinstance(layer, layers.BatchNormalization):
        layer.trainable = False

# Recompile with a much smaller learning rate for fine-tuning
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history_fine = model.fit(
    x_train, y_train,
```

```
    validation_split=0.2,
    epochs=3,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/3
250/250 ━━━━━━━━━━━━━━━━━━━━ 111s 419ms/step - accuracy: 0.9202 - loss: 0.2321 - val_accuracy: 0.8610 - val_loss: 0.4559
Epoch 2/3
250/250 ━━━━━━━━━━━━━━━━━━━━ 105s 419ms/step - accuracy: 0.9349 - loss: 0.1922 - val_accuracy: 0.8720 - val_loss: 0.4433
Epoch 3/3
250/250 ━━━━━━━━━━━━━━━━━━━━ 134s 386ms/step - accuracy: 0.9481 - loss: 0.1519 - val_accuracy: 0.8655 - val_loss: 0.4672
```

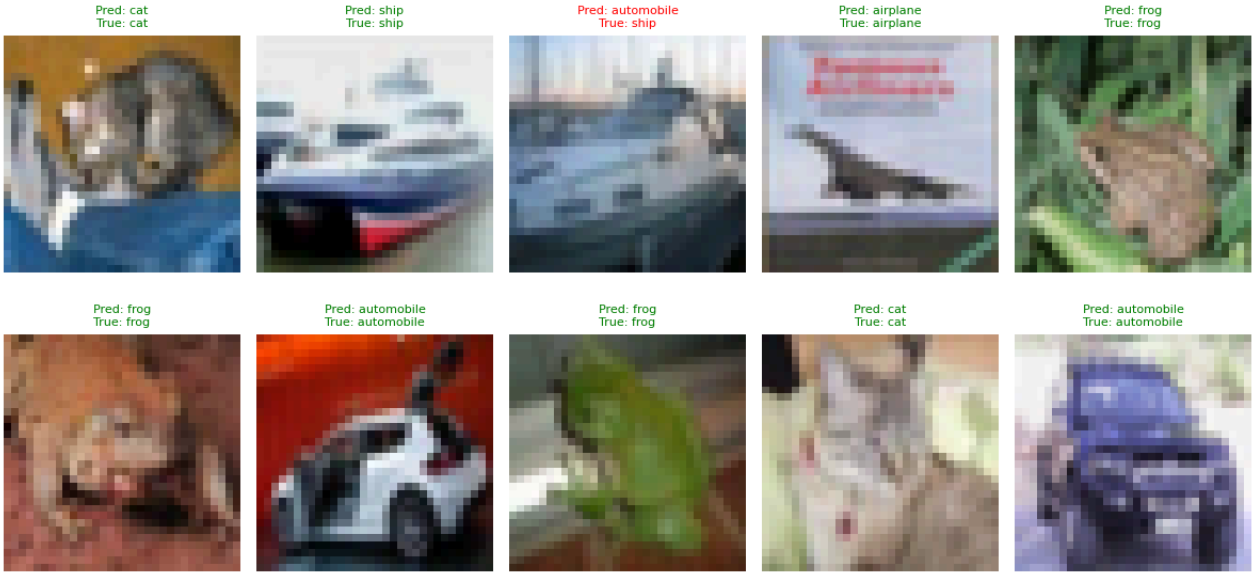
f. Evaluate performance and display predictions

```
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print(f"Test loss: {test_loss:.4f}")
print(f"Test accuracy: {test_accuracy * 100:.2f}%")

probabilities = model.predict(x_test[:10], verbose=0)
predicted_labels = np.argmax(probabilities, axis=1)

plt.figure(figsize=(12, 6))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i])
    color = "green" if predicted_labels[i] == y_test[i] else "red"
    plt.title(f"Pred: {class_names[predicted_labels[i]]}\nTrue: {class_names[y_test[i]]}",
              color=color, fontsize=8)
    plt.axis("off")
plt.tight_layout()
plt.show()
```

```
Test loss: 0.4280
Test accuracy: 86.35%
```

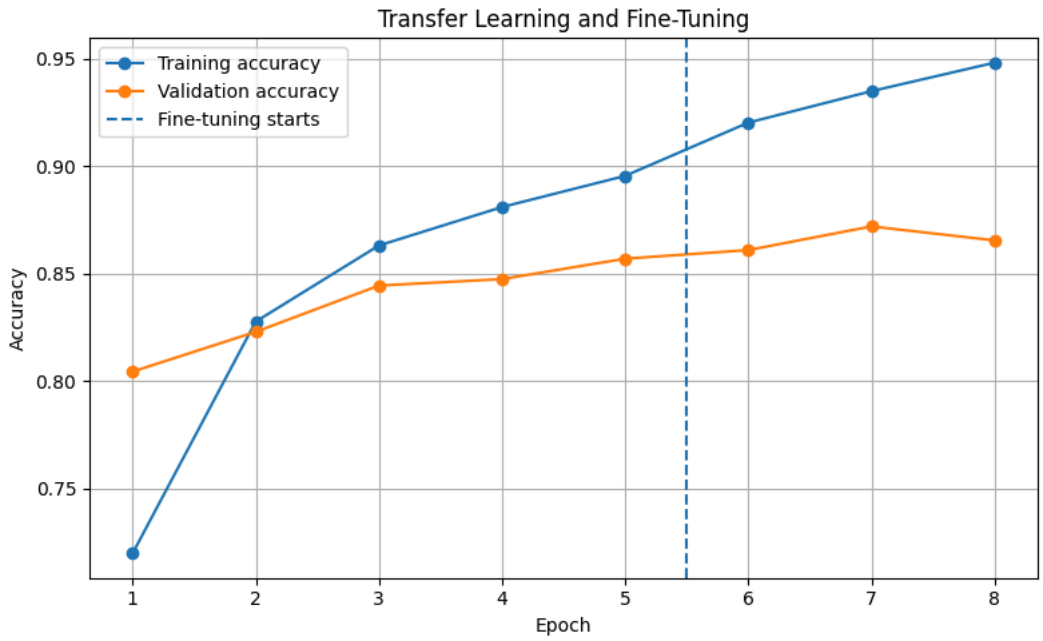


Plot training and validation accuracy

```
train_acc = history_initial.history["accuracy"] + history_fine.history["accuracy"]
val_acc = history_initial.history["val_accuracy"] + history_fine.history["val_accuracy"]

plt.figure(figsize=(8, 5))
plt.plot(range(1, len(train_acc) + 1), train_acc, marker="o", label="Training accuracy")
plt.plot(range(1, len(val_acc) + 1), val_acc, marker="o", label="Validation accuracy")
plt.axvline(len(history_initial.history["accuracy"]) + 0.5, linestyle="--",
            label="Fine-tuning starts")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Transfer Learning and Fine-Tuning")
plt.legend()
```

```
plt.grid(True)
plt.tight_layout()
plt.show()
```



Conclusion

MobileNetV2 pre-trained on ImageNet was used as a feature extractor. The pre-trained layers were initially frozen, a custom classifier was trained, and selected upper layers were unfrozen for fine-tuning with a lower learning rate.

Note: This practical demonstrates transfer learning for image classification. For actual object detection, use a bounding-box dataset and an object-detection architecture.